Comparing Black-Box, White-Box, and Usage-Based Testing Techniques on Perch: Progress Report

Jayden Manyrath and Koen Gennis

CS 7314/5314, Fall 2026 | Prof. Jeff Tian | Project Progress Report | October 29, 2026

1. Summary of the Proposal and Changes in Scope

We proposed to apply four formal techniques to Perch, an open-source Next.js and Supabase application that helps interns find sublets, friends and events: FSM/BBT on the booking and onboarding workflows, PT/BBT on the affordability and verdict rules, CFT/WBT on the rule, expiry and fallback code, and Markov-OP/UBST over a projected usage profile. We compare them on defects, cost, coverage and overlap, using the existing Vitest suite as the baseline. By this report we planned to have built and run the FSM, PT and CFT suites, to have fixed and retested the defects they found, and to have built the Markov chain.

The code walkthrough led to five changes. (1) The implemented vocabulary differs from our proposal: "pending" is `requested`, "confirmed" is `booked`, and a separate listing status (`available`, `taken`, `stale`) carries "taken" and "expired". A state is therefore the pair (booking status, listing status). (2) With no live keys, the app runs on an in-memory fixture data source, a second implementation of the same state machine. We ran the FSM model against both it and the live API path. (3) For FSM, PT and CFT we used an in-memory database fake that also enforces the one-live-hold unique index, which keeps runs deterministic. Live Supabase is kept for the usage-based runs. (4) CFT targets the freshness pass and its cron route, the booking transition code, and the fixture/live fallback in `lib/data/source.ts`. The verdict rules are covered by PT with branch coverage measured. (5) The volunteer pilot slipped one week. The Markov chain is built and validated on the projected profile.

2. Progress So Far

Baseline. The existing suite has 499 cases (492 pass, 7 skipped because they need live keys), runs in about 12 seconds, and reaches 76.7% branch coverage on the 14 target modules. It

never executes the expiry cron route, the server-side `performTransition`, or the booking fallback paths. It passes completely on the original code, so it detected none of the defects below.

Models. The FSM booking model has 7 states and 13 valid transitions. From it we derived 18 invalid "sneak path" pairs (each must be rejected with 403 or 409 and leave state unchanged), 5 multi-event paths and 2 concurrency scenarios. The onboarding model has 8 states, from offer upload to done, with a guard on leaving the review screen. For PT we partitioned 7 input dimensions into valid and invalid classes, with cases on both sides of every boundary, and computed all expected values by hand from the documented constants. For CFT we drew 6 control flow graphs. The Markov chain has 10 screen states plus an absorbing EXIT state.

Execution. Round 1 ran against the unmodified code. Round 2 ran after the fixes (Table 1).

Technique	Cases	Round 1 pass / fail	Round 2 pass / fail	Defects
FSM, live API path	40	38 / 2	40 / 0	D01, D02
FSM, fixture source	19	11 / 8	19 / 0	D03-D06
FSM, onboarding (browser)	10	8 / 1, 1 obs.	9 / 0, 1 obs.	D08
PT	78	68 / 10	78 / 0	D09-D12
CFT	37	33 / 4	37 / 0	D13
Baseline suite	499	492 / 0	492 / 0	none

Table 1. Suites and results by round.

The 25 round-1 failures traced to 12 faults. One more was found while building the FSM model (D07) and one during the walkthrough (D14). The two most severe are D09 and D13. PT found D09: "$45.00 per hour" was parsed as a $45 annual salary with high confidence, which zeroes take-home pay and fails every listing on budget. Intern offers are often hourly, so this is a common input class. CFT found D13: when the live server refused a booking (409 already held, 403 own listing), the client took the outage edge, fell back to the fixture and showed a fabricated "requested" booking.

FSM found five more defects in the booking workflow:

- A taken listing could be relisted while its booking was still booked (D01).
- The loser of two simultaneous requests received HTTP 500 instead of 409 (D02).
- The fixture source let booked bookings be re-approved or declined, and let held, taken or stale listings be requested again (D03-D06).

FSM model construction also showed that the specified booker-cancel transition has no route (D07), and the browser run showed that onboarding accepts an end date before the start date (D08). PT also found that the negotiate route discarded a stated budget lower than the salary ceiling (D10) and accepted malformed or inverted dates (D11). The lease scout printed "Gap of NaN days" (D12). By ODC type the 14 defects are 8 Checking, 2 Algorithm, 2 Assignment, 1 Interface and 1 Function. Twelve were fixed in three commits and retested. D07 (a missing feature) and D14 (hard-coded stay dates on the negotiate screen) remain open.

Analysis so far. Each technique found defects of the kind its model targets:

- FSM's defects all depend on event order or two interacting actors.
- PT's 19 hand-computed tax and rent values all matched the code, but its defects sat in the unusual and invalid classes.
- CFT alone reached the internal fallback edge behind D13, which no black-box oracle can observe.

FSM found the most defects (8 including D07), PT found 4 and CFT found 1. Weighted by severity, PT and CFT each found one of the two high-severity defects. No defect was found by two techniques, which supports using several. Against the baseline (0 defects), the course suites found 13 and raised target-module branch coverage from 76.7% to 83.3% (cron route 0% to 88%, `lib/bookings.ts` 76% to 86%). The 174 new cases run in under 2 seconds and the browser run takes about a minute, so model construction dominated the cost. Hours per technique are being collected in `effort-log.csv` for a defects-per-hour comparison. Failing cases fell from 25 in round 1 to 0 in round 2, and no existing test regressed. The Markov chain passes its validation checks (stochastic rows, absorbing EXIT, analytic expected visits matching 20,000 simulated walks), and the generator has drawn a reproducible 200-walk UBST suite. The

projected profile predicts 5.3 screens per session, with Feed (1.5 visits) and Perches (1.2) dominating.

3. Remaining Work and Schedule

Dates	Remaining work
Oct 30 - Nov 6	Add opt-in navigation logging, run the pilot with 10-15 volunteers, re-estimate the transition matrix from counts
Nov 2 - Nov 13	Build a browser executor for the generated walks against live Supabase; run several UBST rounds with fixes between them; estimate reliability per round; fit a growth model; sensitivity check on the booking and abandonment probabilities
By Nov 13	Resolve D07 and D14 with instructor feedback; complete the effort log
By Nov 20	TA demo: models, both test rounds, defect log, coverage reports
Nov 21 - Dec 8	Cross-technique analysis (applicability, effectiveness, cost, overlap, ODC, reliability trend); add the suites to GitHub Actions as regression tests; final report

All models, suites, raw results and the defect log are in the repository under `docs/testing` and `tests/course`.